

SQL Assignment — Group 4: Influencers and Posts - JOINS

Group 4 — Influencers & Posts: JOINS

1. CREATE influencers + posts, with sample data

Create two tables: *influencers* (*influencer_id* PRIMARY KEY, *name*) and *posts* (*post_id* PRIMARY KEY, *influencer_id* FOREIGN KEY, *caption*). Insert at least 3 influencers and 2 posts for each.

```
CREATE TABLE influencers (  
    influencer_id INT AUTO_INCREMENT PRIMARY KEY,  
    name          VARCHAR(100)  
);  
  
CREATE TABLE posts (  
    post_id          INT AUTO_INCREMENT PRIMARY KEY,  
    influencer_id INT,  
    caption          VARCHAR(255),  
    FOREIGN KEY (influencer_id) REFERENCES influencers(influencer_id)  
);  
  
INSERT INTO influencers (name) VALUES  
    ('Riya Sharma'),  
    ('Karan Mehta'),  
    ('Aisha Khan');  
  
-- influencer_id 1 = Riya, 2 = Karan, 3 = Aisha (based on insert order)  
INSERT INTO posts (influencer_id, caption) VALUES  
    (1, 'Exploring the hills this weekend 🏔️'),  
    (1, 'New travel vlog is live!'),  
    (2, 'Best street food in town 🍛'),  
    (2, 'Recipe of the week: butter chicken'),  
    (3, 'Leg day complete 💪');  
  
-- Note: Aisha has only 1 post here; add a second to satisfy "2 posts each":  
INSERT INTO posts (influencer_id, caption) VALUES  
    (3, 'Morning yoga routine 🧘');
```

Explanation: The `FOREIGN KEY (influencer_id) REFERENCES influencers(influencer_id)` line in the `posts` table links each post to the influencer who made it, and ensures every post's `influencer_id` must correspond to a real row in `influencers`. Each influencer ends up with exactly 2 posts.

2. INNER JOIN — captions with influencer names

Display each post's caption along with the name of the influencer who posted it, using `INNER JOIN`.

```
SELECT p.caption, i.name  
FROM posts p  
INNER JOIN influencers i ON p.influencer_id = i.influencer_id;
```

Explanation: `INNER JOIN` matches rows from `posts` and `influencers` where `p.influencer_id = i.influencer_id`, returning only posts that have a matching influencer (which, given the foreign key constraint, is all of them). The table aliases `p` and `i` keep the query concise.

3. LEFT JOIN — all influencers, including those with no posts

Show all influencers and their posts, including influencers who haven't posted anything. If an influencer has no posts, display 'No Posts' in the caption column.

```
SELECT i.name, IFNULL(p.caption, 'No Posts') AS caption
FROM influencers i
LEFT JOIN posts p ON i.influencer_id = p.influencer_id;
```

Explanation: A `LEFT JOIN` keeps every row from `influencers` (the "left" table) even if there's no matching row in `posts` — for an influencer with no posts, the joined `caption` column would be `NULL`. `IFNULL(p.caption, 'No Posts')` replaces that `NULL` with the literal text 'No Posts' in the output (MySQL's `IFNULL`; the standard-SQL equivalent is `COALESCE(p.caption, 'No Posts')`, which also works in MySQL).

4. RIGHT JOIN — all posts with influencer names

List all posts and the corresponding influencer's name, using `RIGHT JOIN`, ensuring even posts without a matching `influencer_id` are shown.

```
SELECT p.caption, i.name
FROM influencers i
RIGHT JOIN posts p ON i.influencer_id = p.influencer_id;
```

Explanation: `RIGHT JOIN` keeps every row from the "right" table (`posts`) regardless of whether it has a match in `influencers`. If a post existed with an `influencer_id` that doesn't exist in `influencers` (not possible here due to the foreign key, but possible in tables without one), its `name` would show as `NULL` rather than the row being excluded.